LazyStack SLM Assistant Guide

Architecture, Model Selection & Implementation Blueprint for Local Scaffolding Intelligence

This guide provides a comprehensive architecture and implementation reference for integrating a local Small Language Model (SLM) assistant into **LazyStack**. The goal is to let developers describe a stack in plain language—such as *"create a MERN-style app, use PostgreSQL instead of MongoDB, and run the API as a separate service"*—and produce a validated, reviewable blueprint before scaffolding.

1. System Architecture Flow



2. Model Selection Benchmarks

To run reliably on developer workstations without interfering with Docker or IDEs, target models must balance parameter footprint against structured output adherence.

Model	Params	Quant	RAM/VRAM	JSON Reliability	Primary Use Case
Qwen2.5-Coder-1.5B	1.54B	Q4_K_M	~1.2 GB	High	Low-end dev laptops / CPU fallback
Qwen2.5-Coder-3B	3.09B	Q4_K_M	~2.2 GB	**Very High**	**Recommended Default** for LazyStack
Llama-3.2-3B-Instruct	3.21B	Q4_K_M	~2.4 GB	Moderate-High	Conversational clarification turns
DeepSeek-R1-Distill-1.5B	1.78B	Q4_K_M	~1.4 GB	Moderate	Complex constraint reasoning

3. Blueprint JSON Schema Specification

The SLM must output structured JSON matching LazyStack's configuration specifications. Defining a strict Zod schema prevents model hallucinations of non-existent frameworks or invalid ports.

```
import { z } from 'zod';

export const SupportedFrameworks = z.enum([
  'react-ts', 'vue-ts', 'express-js', 'express-ts',
  'nest-js', 'go-fiber', 'go-gin', 'fastapi', 'flutter'
]);

export const SupportedDatabases = z.enum([
  'postgresql', 'mongodb', 'mysql', 'redis', 'sqlite'
]);

export const ServiceBlueprintSchema = z.object({
  name: z.string().regex(/^[a-z0-9-]+$/),
  framework: SupportedFrameworks,
  port: z.number().int().min(1024).max(65535),
  isPublic: z.boolean(),
});

export const DatabaseBlueprintSchema = z.object({
  type: SupportedDatabases,
  instanceName: z.string().regex(/^[a-z0-9-]+$/),
  port: z.number().int().min(1024).max(65535),
  volumeName: z.string(),
});

export const LazyStackBlueprintProposal = z.object({
  action: z.enum(['PROPOSE_BLUEPRINT', 'NEED_CLARIFICATION']),
  clarificationQuestion: z.string().optional(),
  projectName: z.string().regex(/^[a-z0-9-]+$/),
  architecture: z.enum(['monolith', 'modular_monolith', 'microservices']),
  services: z.array(ServiceBlueprintSchema),
  databases: z.array(DatabaseBlueprintSchema),
  proxy: z.enum(['nginx', 'caddy', 'none']),
  dockerCompose: z.boolean(),
  reasoningSummary: z.string().max(300),
});

export type LazyStackBlueprint = z.infer<typeof LazyStackBlueprintProposal>;
```

4. Orchestration & Validation Pipeline

The orchestrator interfaces with Ollama, grounds the model with active catalog constraints, and performs self-correction loops if validation fails.

```
import { LazyStackBlueprintProposal, type LazyStackBlueprint } from './schema';

export async function generateBlueprint(
  userPrompt: string,
  catalog: Record<string, unknown>,
  retryCount = 0
): Promise<LazyStackBlueprint> {
  const MAX_RETRIES = 2;

  const response = await fetch('http://localhost:11434/api/chat', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({
      model: 'qwen2.5-coder:3b',
      stream: false,
      format: 'json',
      messages: [
        { role: 'system', content: buildSystemPrompt(catalog) },
        { role: 'user', content: userPrompt }
      ],
      options: { temperature: 0.1, num_ctx: 8192 }
    }),
  });

  const data = await response.json();
  try {
    const parsed = JSON.parse(data.message.content);
    const validated = LazyStackBlueprintProposal.parse(parsed);
    validatePortCollisions(validated);
    return validated;
  } catch (error) {
    if (retryCount < MAX_RETRIES) {
      const repairPrompt = `${userPrompt}\n\nPrevious attempt failed validation: ${error}. Fix
the JSON output.`;
      return generateBlueprint(repairPrompt, catalog, retryCount + 1);
    }
    throw new Error(`Blueprint generation failed after retries: ${error}`);
  }
}
```

5. Security & Threat Mitigation Matrix

Interacting with local disk scaffolding requires strict boundary defenses against prompt injection and directory traversal attacks.

Threat Vector	Risk Vulnerability	Mandatory Mitigation
Prompt Injection	User text contains malicious instructions attempting shell execution or file overwrites.	Sanitize inputs; enforce Zod schema parsing. Never pass unvalidated string outputs directly to system shell commands.
Path Traversal	SLM generates service or folder names containing relative paths (e.g., `../../etc/passwd`).	Strict regex enforcement (`/^[a-z0-9-]+$/`) on all path, service, database, and project names.
Unauthorized Scaffolding	SLM immediately triggers file system changes or provisions Docker containers without consent.	Mandatory Human-in-the-Loop (HITL) step requiring explicit user approval before execution.

6. Implementation Roadmap

Phase	Key Objectives	Deliverables
Phase 1	Core Foundation & Runtime Abstraction	Zod schema definition, Ollama/llama.cpp provider interface, local health checks.
Phase 2	Orchestration & Repair Loop	Catalog grounding prompt, multi-turn clarification engine, 1-step repair loop.
Phase 3	UI Integration & Safety	Visual Diff Preview UI, HITL approval trigger, fallback states for stopped runtimes.